

SIMATS ENGINEERING ASSESSMENT TOOL3-VIVA QUESTIONS

1. Open Services on a Windows System

Question

How would you evaluate the security posture of a Windows system after identifying multiple open services?

Answer

I would first identify the open ports and running services using a network scanner. Then I would determine the service versions and check whether they have known vulnerabilities. Unnecessary or outdated services should be disabled or patched. The risk is higher when sensitive services are exposed to untrusted networks.

Implementation

Run in Kali:

```
nmap -sV 127.0.0.1
```

Expected output

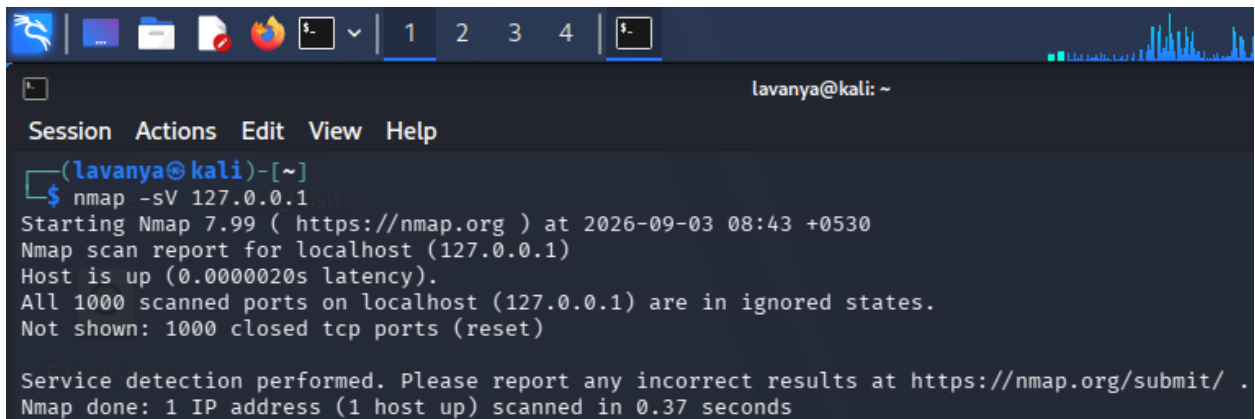
You may see something like:

PORT	STATE	SERVICE	VERSION
------	-------	---------	---------

22/tcp	open	ssh	OpenSSH
--------	------	-----	---------

80/tcp	open	http	Apache
--------	------	------	--------

+



```
lavanya@kali: ~  
Session Actions Edit View Help  
(lavanya@kali)-[~]  
$ nmap -sV 127.0.0.1  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-03 08:43 +0530  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.0000020s latency).  
All 1000 scanned ports on localhost (127.0.0.1) are in ignored states.  
Not shown: 1000 closed tcp ports (reset)  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 0.37 seconds
```

2. Exposed RPC Service

Question

How would you evaluate the security risk associated with an exposed RPC service?

Answer

I would identify RPC-related ports and determine which services are running on them. I would check whether the service is necessary, properly configured and patched. An exposed RPC service may allow unauthorized users to interact with system functions, increasing the risk of information disclosure or unauthorized access.

Implementation

`nmap -sV -p 135,139,445 127.0.0.1`

Expected output

PORT STATE SERVICE

135/tcp open msrpc

139/tcp closed netbios-ssn

445/tcp closed microsoft-ds

Your output may be different depending on your system.

```
(lavanya@kali)-[~]
$ chmod 644
chmod: missing operand after '644'
Try 'chmod --help' for more information.

(lavanya@kali)-[~]
$ curl http://127.0.0.1
curl: (7) Failed to connect to 127.0.0.1 port 80 after 0 ms: Could not connect to server

(lavanya@kali)-[~]
$ nmap -sV -p 135,139,445 127.0.0.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-03 08:46 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000089s latency).

PORT      STATE SERVICE      VERSION
135/tcp    closed msrpc
139/tcp    closed netbios-ssn
445/tcp    closed microsoft-ds

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.21 seconds
```

3. Unnecessary Services in Linux

Question

How would you evaluate the security risks of unconnected or unnecessary services running on a Linux system?

Answer

I would list the active services and identify services that are not required.

Unnecessary services increase the attack surface because each running service may

contain vulnerabilities or configuration weaknesses. Unused services should be disabled and regularly monitored.

Implementation

First run:

```
systemctl --type=service --state=running
```

Then:

```
ss -tuln
```

Expected output

```
LISTEN 0 128 0.0.0.0:22
```

```
LISTEN 0 128 127.0.0.1:631
```

```
lavanya@kali: ~  
Session Actions Edit View Help  
(lavanya@kali)-[~]  
$ nmap -sV 127.0.0.1  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-03 08:43 +0530  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.0000020s latency).  
All 1000 scanned ports on localhost (127.0.0.1) are in ignored states.  
Not shown: 1000 closed tcp ports (reset)  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 0.37 seconds  
  
(lavanya@kali)-[~]  
$ systemctl  
UNIT                                LOAD  
proc-sys-fs-binfmt misc.automount  loaded  
sys-devices-pci0000:00-0000:00:01.1-ata1-host0-target0:0:0-0:0:0-block-sr0.device loaded  
sys-devices-pci0000:00-0000:00:03.0-net-eth0.device loaded  
sys-devices-pci0000:00-0000:00:05.0-sound-card0-controlC0.device loaded  
sys-devices-pci0000:00-0000:00:0d.0-ata3-host2-target2:0:0-2:0:0-block-sda-sda1.device loaded  
sys-devices-pci0000:00-0000:00:0d.0-ata3-host2-target2:0:0-2:0:0-block-sda-sda2.device loaded  
sys-devices-pci0000:00-0000:00:0d.0-ata3-host2-target2:0:0-2:0:0-block-sda-sda5.device loaded  
sys-devices-pci0000:00-0000:00:0d.0-ata3-host2-target2:0:0-2:0:0-block-sda.device loaded  
sys-devices-platform-serial8250-serial8250:0-serial8250:0.0-tty-ttyS0.device loaded  
sys-devices-platform-serial8250-serial8250:0-serial8250:0.1-tty-ttyS1.device loaded  
sys-devices-platform-serial8250-serial8250:0-serial8250:0.2-tty-ttyS2.device loaded  
sys-devices-platform-serial8250-serial8250:0-serial8250:0.3-tty-ttyS3.device loaded  
sys-devices-virtual-misc-rfkill.device loaded  
sys-module-configfs.device loaded  
sys-module-fuse.device loaded  
sys-subsystem-net-devices-eth0.device loaded  
-.mount loaded  
dev-hugepages.mount loaded  
dev-mqueue.mount loaded  
proc-sys-fs-binfmt_misc.mount loaded  
run-rpc_pipefs.mount loaded  
run-user-1000-doc.mount loaded  
run-user-1000-gvfs.mount loaded  
run-user-1000.mount loaded  
sys-fs-fuse-connections.mount loaded  
sys-kernel-config.mount loaded  
sys-kernel-debug.mount loaded  
sys-kernel-tracing.mount loaded  
  
(lavanya@kali)-[~]  
$ ss -tuln  
Netid      State      Recv-Q     Send-Q     Local Address:Port
```

Conclusion

The results show which services are actively running and which network ports are listening. Services that are not required should be disabled to reduce the attack surface.

4. Incorrect Linux File Permissions

Question

A Linux file has incorrect permissions that allow unauthorized modification. How would you evaluate the resulting security risk?

Answer

I would inspect the file ownership and permissions using `ls -l`. If unauthorized users have write permission, they may modify the file and potentially affect system operations or application behavior. I would correct the permissions and verify them again.

Implementation

Create a test file:

```
echo "Security Test" > testfile.txt
```

Check permissions:

```
ls -l testfile.txt
```

Give everyone write permission for demonstration:

```
chmod 666 testfile.txt
```

Check again:

```
ls -l testfile.txt
```

Expected output

Initially:

```
-rw-r--r-- 1 user user ... testfile.txt
```

After `chmod 666`:

```
-rw-rw-rw- 1 user user ... testfile.txt
```

The `rw` permissions for others show that other users can modify the file.

Then fix it:

```
chmod 644 testfile.txt
```

Verify:

```
ls -l testfile.txt
```

Conclusion

Weak file permissions can allow unauthorized modification. Restricting write access reduces the possibility of tampering.

```
(lavanya@kali)-[~]  
$ chmod 666  
chmod: missing operand after '666'  
Try 'chmod --help' for more information.  
  
(lavanya@kali)-[~]  
$ chmod 644  
chmod: missing operand after '644'  
Try 'chmod --help' for more information.
```

5. SQL Injection in a Web Application

Question

A web application is vulnerable to SQL injection. How would you evaluate the severity of this vulnerability and its possible consequences?

Answer

I would determine whether user input is directly included in SQL queries without proper validation or parameterization. I would assess whether the vulnerability could expose, modify or delete database information. Severity depends on the sensitivity of the data, authentication requirements and the level of access obtainable. Parameterized queries and input validation should be used to prevent SQL injection.

Safe local implementation

If you already have a local web application for testing, check its login/input fields using a controlled test environment.

For example, first verify that your local web server is running:

```
curl http://127.0.0.1
```

Expected output

You should receive HTML content such as:

HTML

```
<html>
```

...

```
</html>
```

If you are using DVWA/OWASP Juice Shop, you can perform the SQL-injection experiment inside that intentionally vulnerable lab instead of testing a real website.

```
(lavanya@kali)-[~]
$ sudo systemctl start apache2

(lavanya@kali)-[~]
$ curl http://127.0.0.1

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Debian Default Page: It works</title>
    <style type="text/css" media="screen">
      * {
        margin: 0px 0px 0px 0px;
        padding: 0px 0px 0px 0px;
      }

      body, html {
        padding: 3px 3px 3px 3px;
        background-color: #D8DBE2;
        font-family: Verdana, sans-serif;
        font-size: 11pt;
        text-align: center;
      }

      div.main_page {
        position: relative;
        display: table;
        width: 800px;
        margin-bottom: 3px;
        margin-left: auto;
        margin-right: auto;
        padding: 0px 0px 0px 0px;
        border-width: 2px;
        border-color: #212738;
        border-style: solid;
        background-color: #FFFFFF;
        text-align: center;
      }

```